

Aula 4 - Matrizes

Essa aula se resumiu a estudar como simulamos **matrizes** em C++.

Vetores de vetores

Anteriormente, eu citei que os vetores (tanto arrays quanto vectors) têm a capacidade de guardar qualquer tipo de variável.

```
int array[10];
char array2[20];
float array3[30];
...
vector<int> a;
vector<char> b;
vector<float> c;
...
```

Daí, podemos extrapolar essa ideia: e se fizéssemos **vetores de vetores**?

Isso, embora difícil de entender de cara, é basicamente a representação de uma matriz em C++. Podemos declarar uma de duas formas principais:

```
vector<vector<int>> matriz; // usando vectors

int matriz[n][m]; // usando arrays
// esse último, precisamos especificar o número n de linhas d
a matriz
// e m de colunas
```

Mais uma vez, observamos que não conseguimos usar arrays sem declarar seu tamanho. Já com vectors, isso não é obrigatório.

Note que, se tentarmos acessar o `matriz[i]`, não estaremos acessando um inteiro, mas sim um vetor de inteiros na i-ésima posição. Então, para acessar um

elemento propriamente dito, precisamos tratar `matriz[i]` como um vetor e colocar outro índice:

```
matriz[i][j];
```

Formalmente, isso corresponde a: "o j -ésimo elemento, do i -ésimo vetor, do vetor chamado 'matriz'."

Informalmente, dada uma matriz de dimensões $n \times m$:

$$M = \begin{bmatrix} m_{00} & m_{01} & \cdots & m_{0\ m-1} \\ m_{10} & m_{11} & \cdots & m_{1\ m-1} \\ \vdots & \vdots & \ddots & \vdots \\ m_{n-1\ 0} & m_{n-1\ 1} & \cdots & m_{n-1\ m-1} \end{bmatrix}$$

`matriz[i][j]` é o elemento m_{ij} . O elemento na linha i e coluna j .

Observação importante: como ainda estamos mexendo com vetores, obviamente, os índices também vão de 0 até `tamanho - 1`.

Manipulando entradas e saídas

Daí, surge um problema. Digamos que um problema no beecrowd diz que você precisa receber uma matriz de inteiros. Como é feita a entrada nessa situação?

Usamos **loops aninhados** (um loop dentro do outro):

```
for(int i = 0; i<n; ++i){
    for(int j = 0; j<m; ++j){
        cin >> matriz[i][j];
    }
}
```

E, se quisermos imprimir uma matriz (de forma esteticamente atraente), precisamos fazer algo similar:

```
for(int i = 0; i<n; ++i){
    for(int j = 0; j<m; ++j){
```

```

        cout << matriz[i][j] << " ";
    }
    cout << endl;
}

```

Convido vocês a testarem esses loops aninhados com alguma matriz pequena, tipo 3×3 ou 2×2.

Particularidades dos vectors

Notem que ao declarar uma matriz dessa forma:

```
vector<vector<int>> matriz;
```

Ela vem vazia. Via de regra, quando trabalhamos com vectors, iniciamos eles vazios e usamos o `push_back()` para preenchê-los. Mas como funciona o `push_back()` aqui? Como cada `matriz[i]` é um vetor por si só, temos que usar:

```
matriz[i].push_back(x);
```

Formalmente: isso coloca o elemento x no final do vetor na posição i do vetor chamado "matriz".

Informalmente, isso coloca um elemento x na última posição ainda não preenchida da linha i da matriz.

Isso fica chatíssimo de lidar com loops aninhados. Então, a solução pra simplificar nossa vida poderia ser declarar o tamanho dos vectors previamente, aí o loop aninhado que eu mostrei antes funcionaria perfeitamente para receber a matriz inteira como entrada. Como faríamos isso?

Se, com vectors de inteiros comuns, ara declararmos um vector com tamanho previamente definido, e com um valor padrão previamente definido (opcional), fazíamos:

```
vector<int> exemplo(tamanho, valor_padrao);
```

Com matrizes, precisamos pensar em “declarar um vector com tamanho N, e essas N posições têm que estar preenchidas com outros vectors de tamanho M”. Isso se traduz para:

```
vector<vector<int>> matriz(n, vector<int>(m));  
// ou, se quisermos inicializar a matriz inteira com um valor  
padrao  
vector<vector<int>> matriz(n, vector<int>(m, valor_padrao));
```

Curiosidades e cuidados

1. Eu aconselho vocês a usarem **arrays** quando forem mexer com matrizes. Isso é porque raramente precisamos de matrizes de tamanho variável (isso vai ser útil daqui a algum tempo, quando entrarmos em Algoritmos em Grafos, mas agora não), então, o vector acaba sendo um overkill desnecessário. Sem falar que, dadas as particularidades de matrizes com vectors citadas acima, convenhamos que os arrays são bem mais simples de decorar e usar.
2. Vale ressaltar que, se conseguimos criar vetores de qualquer coisa, também conseguimos criar um vetor de vetores de vetores de inteiros (ou um vetor de matrizes). Isso nos dá a ideia de “matriz tridimensional”:

```
int matriz[10][10][10];  
vector<vector<vector<int>>> matriz;
```

E isso pode ser explodido para outras dimensões também. Isso é útil em problemas de **Programação Dinâmica** (aulas futuras...) mais avançados.

3. Se tratávamos strings como vetores de chars, um vetor de strings é, na verdade, uma matriz de chars. Então, podemos brincar com eles de formas similares como fazemos com matrizes de inteiros:

```
string lista_de_nomes[10];  
lista_de_nomes[5][0]; // pega a primeira letra do sexto nome  
na lista
```

4. CURIOSIDADE INÚTIL → podemos declarar matrizes misturando arrays e vectors (não façam isso, mas se verem por aí, já sabem o que significa):

```
vector<int> matriz[10];  
// "vector<int>" é o tipo do array de 10 posições chamado de  
"matriz"
```